

Assign Cookies

$$\text{greed} = [1 \ 5 \ 3 \ 3 \ 4] \quad s = [4 \ 2 \ 1 \ 2 \ 1 \ 3]$$

4 x 3 x x ✓ ✓

here in greed there is greeting factor for each children and s contains size of cookie

If you give to 5 $s=4$ then he will not be satisfied he will only be satisfied for $s \geq 5$

$$\text{greed} = [1 \ 5 \ 3 \ 3 \ 4] \quad s = [4 \ 2 \ 1 \ 2 \ 1 \ 3]$$

1 x 3 4 x ✓ ✓ ✓

3

in

this pattern we are able to satisfy 3 children

[1 1 2 2 3 4]

[1 3 3 4 5]

we will sort both arrays and try to assign the array

```

func (gneed, size)
{
    n = gneed.size    m = size.size();
    l = 0, h = 0
    sort(gneed)    sort(size)

    while (l < m)
    {
        if (gneed[h] <= size[l])
        {
            h =

```

whether size of cookie satisfies or not in both cases we need to move to next cookie

```

while (l < m)
{
    if (gneed[h] <= size[l])
    {
        h = h + 1;
    }
    l = l + 1;
}

```

```

while (l < m)
{
    if (gcd[l] <= size[l])
    {
        n = n + 1;
    }
    l = l + 1;
}

return

```

```

O(N log N + M log M + M)
O(1)

```

this is time complexity and

space complexity

See striver greedy algorithm playlist if has any doubt

455. Assign Cookies

Solved 

Easy

 Topics

 Companies

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: $g = [1,2,3]$, $s = [1,1]$

Output: 1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Example 2:

Input: $g = [1,2]$, $s = [1,2,3]$

 4.8K



 328



 72 Online

```
1 class Solution {
2     public int findContentChildren(int[] g, int[] s) {
3         int n = g.length; //no of children
4         int m = s.length; //no of cookies
5         Arrays.sort(g);
6         Arrays.sort(s);
7         int l=0;
8         int r=0;
9         while(l<m){
10             if(r==n) return n; //all the children got cookies
11             if(g[r]<=s[l]){
12                 //child is satisfied
13                 r++;
14             }
15             l++;
16         }
17         return r;
18     }
19 }
```

[leetcode](https://leetcode.com/problems/assign-cookies/)

860. Lemonade Change

Easy

Topics

Companies

At a lemonade stand, each lemonade costs \$5. Customers are standing in a queue to buy from you and order one at a time (in the order specified by bills). Each customer will only buy one lemonade and pay with either a \$5, \$10, or \$20 bill. You must provide the correct change to each customer so that the net transaction is that the customer pays \$5.

Note that you do not have any change in hand at first.

Given an integer array `bills` where `bills[i]` is the bill the i^{th} customer pays, return `true` if you can provide every customer with the correct change, or `false` otherwise.

Example 1:

Input: `bills = [5,5,5,10,20]`

Output: `true`

Explanation:

From the first 3 customers, we collect three \$5 bills in order.

From the fourth customer, we collect a \$10 bill and give back a \$5.

From the fifth customer, we give a \$10 bill and a

Java ▾ 🔒 Autc

```
1 class Sol
2 |     publi
   bills) {
3
4     }
5 }
```

Saved

👍 3.3K 🗨️ 231 ☆ 📄 ?

● 39 Online

✅ Testcase >



```

bool func (arr)
{
    five = 0, ten = 0
    for (i = 0 -> n)
    {
        if (arr[i] == 5) five = five + 1;
        else if (arr[i] == 10)
        {
            if (five > 0)

```

```

            five--;
            ten++;
        }
        else if (arr[i] == 20)
        {
            if (ten > 0)
            {
                ten--;
                five--;
            }
            else if (five >= 2)
            {
                five--;
                ten++;
            }
            else return false;
        }
    }
    return true;
}

```

we are not updating about 20 rs note as it is of no significance since if someone give me 20 i have to return 15 if 10 then i have to return 5 and if 5

then 0 so there is no use of 20 rs note that i got from customer

```
1 class Solution {
2     public boolean lemonadeChange(int[] bills) {
3         int five = 0;
4         int ten = 0;
5         for(int i=0;i<bills.length;i++){
6             if(bills[i]==5){
7                 five+=1;
8             }
9             else if(bills[i]==10){
10                if(five==0)return false;
11                else{
12                    five-=1;
13                    ten+=1;
14                }
15            }
16            else {
17                //when it is 20 have to return 15
18                if(five!=0 && ten!=0){
19                    five-=1;
20                    ten-=1;
21                }
22                // agar 10 na ho 5 tin ho to tb ham three coins of 5 de skte hai
23                else if(five>=3){
24                    five-=3;
25                }
26            }
27        }
28    }
29 }
```

Saved

```
15 }
16     else {
17         //when it is 20 have to return 15
18         if(five!=0 && ten!=0){
19             five-=1;
20             ten-=1;
21         }
22         // agar 10 na ho 5 tin ho to tb ham three coins of 5 de skte hai
23         else if(five>=3){
24             five-=3;
25         }
26         else{
27             return false;
28         }
29     }
30 }
31 return true;
32 }
33 }
34 }
```

Shortest Job first

Difficulty: **Medium**

Accuracy: **68.79%**

Submissions: **42K+**

Points: **4**

Average Time: **30m**

Geek is a software engineer. He is assigned with the task of calculating **average waiting time** of all the processes by following **shortest job first** policy.

The shortest job first (SJF) or shortest job next, is a scheduling policy that selects the waiting process with the smallest execution time to execute next.

Given an array of integers **bt** of size **n**. Array **bt** denotes the **burst time** of each process. Calculate the **average waiting time** of all the processes and return the nearest integer which is smaller or equal to the output.

Note: Consider all process are available at time 0.

Example 1:

Input:

$n = 5$

$bt = [4, 3, 7, 1, 2]$

Output: 4

Explanation: After sorting burst times by shortest job policy, calculated average waiting time is 4.

t=0

Shortest job first (SJF) → scheduling policy that selects the waiting process with the smallest execution time to execute next.

$bt = [4, 3, 7, 1, 2]$
 $P_1 \ P_2 \ P_3 \ P_4 \ P_5$

Execution timeline:

0 — 1 — 3 — 6 — 10 — 12
 $P_4 \ P_5 \ P_2 \ P_1 \ P_3$

Waiting times:

$P_4 \rightarrow 0$
 $P_5 \rightarrow 1$
 $P_2 \rightarrow 3$
 $P_1 \rightarrow 6$
 $P_3 \rightarrow 10$

$\frac{20}{5} = 4$

When 1 starts its waiting time is 0 when 2 starts its waiting time is 1 when 3 starts its waiting time is $(2+1) = 3$ when 4 starts its waiting time is $(3+3) = 6$ when 7 starts its waiting time is $(6+4) = 10$ so total waiting time of process = $(0+1+3+6+10) = 20$ average waiting time is $20/5 = \text{waitingTime}$

t=0

Shortest job first (SJF) → scheduling policy that selects the waiting process with the smallest execution time to execute next.

bt = [4] [3] [7] [1] [2]
P1 P2 P3 P4 P5

Execution order: [1] [2] [3] [4] [7]

Waiting times: 0, 1, 3, 6, 10

wt Time = 0 + 1 + 3 + 6 + 10 = 20

Average waiting time = $20/5 = 4$



```
2
3 class Solution {
4     static int solve(int bt[]) {
5         // code here
6         int waitingTime =0;
7         int time =0;
8         int n = bt.length;
9         Arrays.sort(bt);
10        for(int i=0;i<n;i++){
11            waitingTime+=time;
12
13            time+=bt[i];
14        }
15        return waitingTime/n;
16    }
17 }
18
```

✓ Total Time Complexity

The dominant term is sorting:

$$O(n \log n)$$

🧠 Space Complexity

Variables used

java

📋 Copy code

```
int waitingTime, time, n;
```

- Constant extra space $\rightarrow O(1)$

Sorting space

- `Arrays.sort(int[])` sorts **in-place**
- Uses $O(\log n)$ stack space due to recursion

✓ Total Space Complexity

$$O(\downarrow n)$$

(If recursion stack is ignored, it is often considered $O(1)$ auxiliary space.)

+ Ask anything



ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).

55. Jump Game

Medium

Topics

Companies

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: `true`

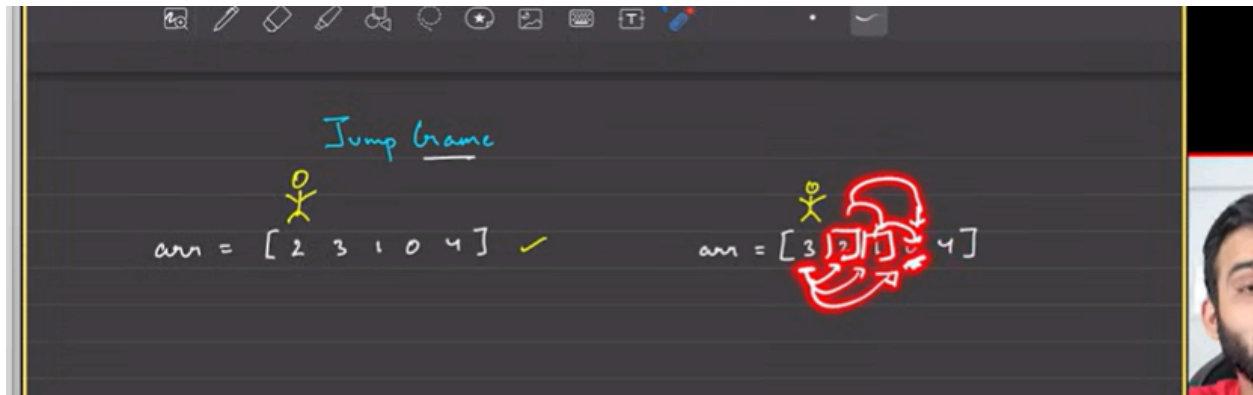
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

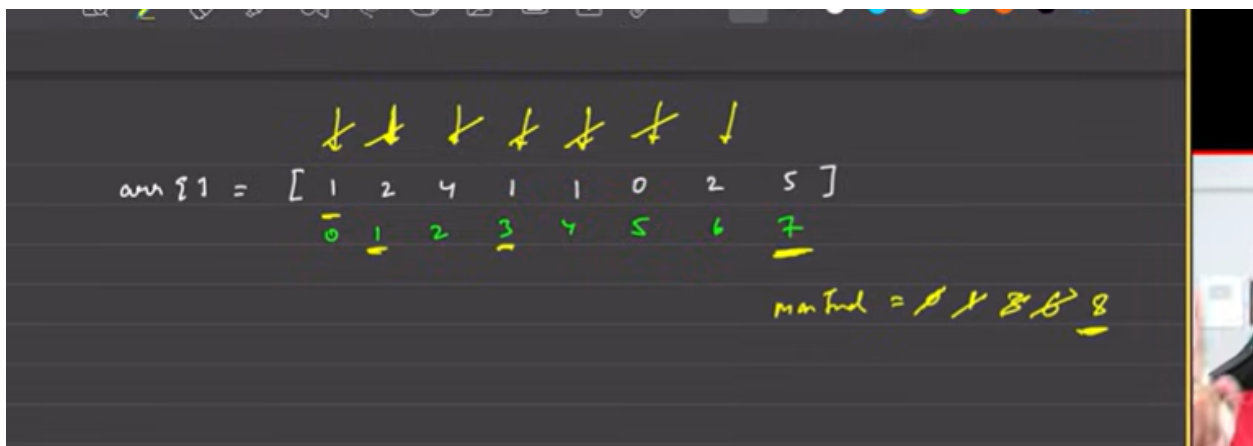
Input: `nums = [3,2,1,0,4]`

Output: `false`

Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

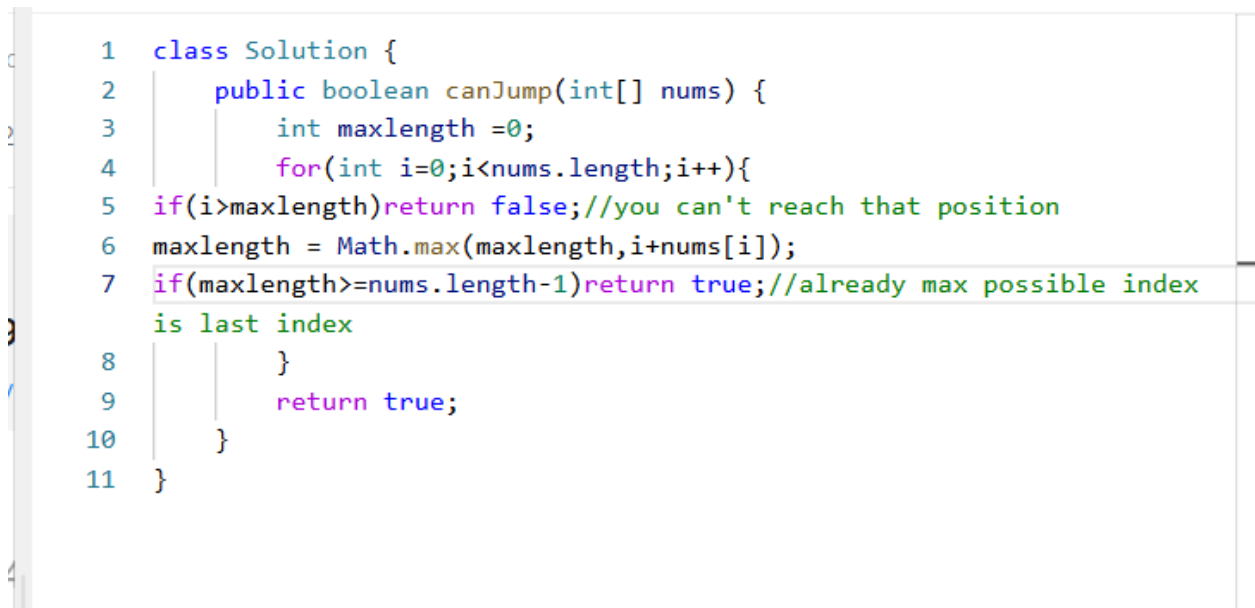
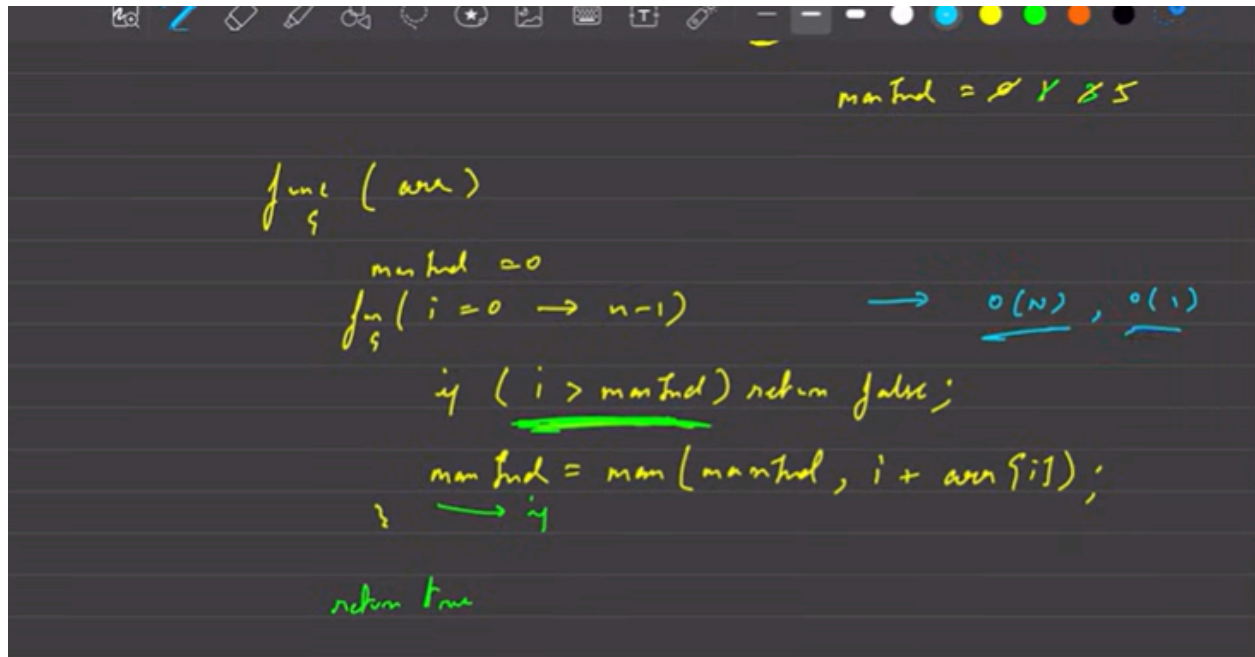


if the arrays doesn't have 0 then it can always go to the last position



below example you can't reach the last one





leetcode time complexity is n and space complexity is 1

45. Jump Game II

Medium

Topics

Companies

You are given a **0-indexed** array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- $0 \leq j \leq \text{nums}[i]$ and
- $i + j < n$

Return the minimum number of jumps to reach index `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2.
Jump 1 step from index 0 to 1, then 3

16.2K | 264 | ☆ | ? | 204 Online

```
arr = { 2 3 1 1 4 }
f(ind, jumps)
if (ind >= n-1) return jumps;
mini = INT_MAX;
for (i = 1 -> arr[ind]);
    mini = min(mini, f(ind + i, jump + 1);
return mini;
```

$$TC \rightarrow N^N$$
$$SC \rightarrow O(N)$$

Time & Space Complexity

 **Time Complexity:** Exponential ($O(N^N)$)

- At each index, you try multiple recursive calls
- Huge overlap of subproblems

 **Space Complexity:** $O(N)$ (recursion stack)

Important Interview Note !

If someone asks:

“How did you come up with this logic?”

Say:

“From each index, I try all possible jumps allowed and recursively compute the minimum jumps needed to reach the end. Since the answer is the minimum among all choices, I take the minimum of all recursive results.”

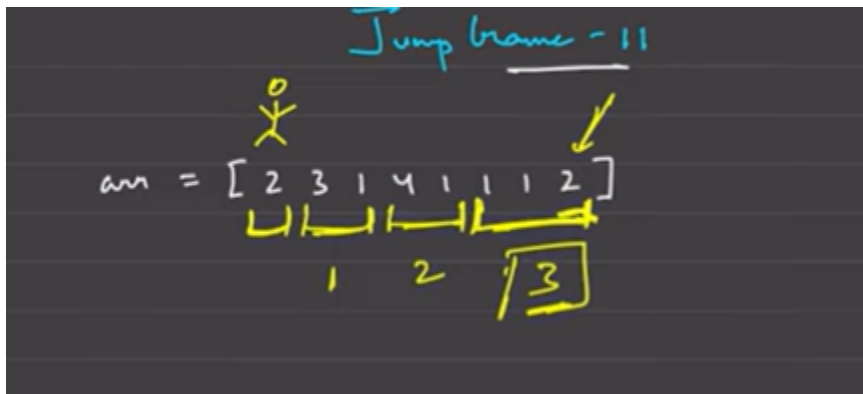
```
Java ▾ 🔒 Auto
1 class Solution {
2     public int minjump(int[] nums,int index,int jump){
3         if(index>=nums.length-1)return jump;
4         int minjumps = Integer.MAX_VALUE;
5         for(int i=1;i<=nums[index];i++){
6             minjumps = Math.min(minjumps,minjump(nums,index+i,jump+1));
7         }
8     }
9     return minjumps;
10 }
11 public int jump(int[] nums) {
12     int jump =0;
13     int ans= minjump(nums,0,0);
14     return ans;
15 }
16 }
17 }
```

Saved

Ln 6, Col 5

optimised solution is using dynamic programming do it after knowing dp

Second optimise approach is using optimised recursion



Jump Game - II

arr = [2, 3, 1, 4, 1, 1, 2]

jumps = 0

Jump Game - II

arr = [2, 3, 1, 4, 1, 1, 2]

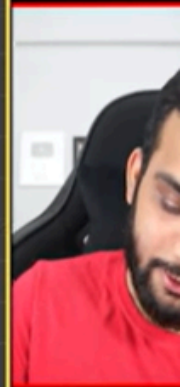
jumps = 0

```

jumps = 0  l = 0  r = 0
while (r < n-1)
{
    farthest = 0
    for (ind = l -> r)
    {
        farthest = max(i + arr[ind], farthest);
    }
    l = r + 1
}

```

```
    jumps = 0  
    while (n < n-1)  
    {  
        farthest = 0  
        for (ind = 1 → n)  
        {  
            farthest = max(i + arr[ind], farthest);  
        }  
        l = n+1      jumps = jumps+1;  
        n = farthest  
    }
```



```
    }  
    return jumps;  
}
```



```
1 class Solution {  
2     public int jump(int[] nums) {  
3         int l=0;  
4         int r=0;  
5         int jump=0;  
6         while(r<nums.length-1){  
7             int farthest =0;  
8             for(int i=l;i<=r;i++){  
9                 farthest = Math.max(farthest,i+nums[i]);  
10            }  
11            l=1+r;  
12            r=farthest;  
13            jump++;  
14        }  
15        return jump;  
16    }  
17 }
```

Complexity (Very Important)

- Time: $O(N)$
- Space: $O(1)$
 - ✓ Optimal solution

Key Takeaway

[chatgpt](#) [leetcode](#)

Job Sequencing Problem



Difficulty: **Medium**

Accuracy: **34.51%**

Submissions: **360K+**

Points: **4**

You are given two arrays: **deadline[]**, and **profit[]**, which represent a set of jobs, where each job is associated with a **deadline**, and a **profit**. Each job takes 1 unit of time to complete, and only one job can be scheduled at a time. You will earn the profit associated with a job only if it is completed by its deadline.

Your task is to find:

1. The **maximum number of jobs** that can be completed within their deadlines.
2. The **total maximum profit** earned by completing those jobs.

Examples :

Input: `deadline[] = [4, 1, 1, 1], profit[] = [20, 10, 40, 30]`

Output: `[2, 60]`

Explanation: Job₁ and Job₃ can be done with maximum profit of 60 (20+40).

Input: `deadline[] = [2, 1, 2, 1, 1], profit[] = [100, 19, 27, 25, 15]`

Output: `[2, 127]`

Explanation: Job₁ and Job₃ can be done with maximum profit of 127 (100+27).

Input: `deadline[] = [3, 1, 2, 2], profit[] = [50, 10, 20, 30]`

Output: `[2, 100]`

Menu



gf9 9s

id	deadline	profit
1	4	20
2	1	10
3	1	40
4	1	30

id → 3 id → 1
+ 40 + 20
= 60

delay the job to the end days

1st job you can perform it on 1st or 2nd or 3rd or 4th day but not on 5th day. 2nd, 3rd, 4th job can only be performed on 1st day.

Each job takes only one day to complete

<u>N</u>	id	deadline	profit ↑
→ 6	2	80	
→ 3	6	70	
→ 4	6	65	
→ 2	5	60	
→ 5	4	25	
→ 8	2	22	
→ 1	4	20	
7	2	10	

1	8	6	5	2	4	3
1	1	1	1	1	1	1
0	1	2	3	4	5	6

countProfit = 0	cnt = 0
80	1
150	2
215	3
275	4
300	5
322	6

```

Jobs {
    id, profit, deadline
}

bool comp (Jobs val1, Jobs val2)
{
    return val1.profit > val2.profit
}
    
```

```
func (jobs arr [])
```

```
    sort(arr, comp); totProfit = 0, cut = 0, maxDeadline = -1
```

```
    for (i = 0 → n-1) { maxDeadline = max(maxDeadline, arr[i].
```

```
        deadline);
```

```
    hash[maxDeadline+1] = -1;
```

```
    for (i = 0 → n-1)
```

```
    { j = arr[i].deadline → 0)
```

```
        if (hash[j] == -1)
```

```
            cut = cut + 1;
```

```
            hash[j] = arr[i].id;
```

```
            totProfit += arr[i].profit;
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
[ for (j = arr[i].deadline → 0) ]
```

```
    if (hash[j] == -1)
```

```
        cut = cut + 1;
```

```
        hash[j] = arr[i].id;
```

```
        totProfit += arr[i].profit;
```

DSU

this

loop can be optimised by using disjoint set taught in graph playlist

◆ Key Greedy Idea (MOST IMPORTANT)

Always try to do the job with the highest profit first, and schedule it as late as possible before its deadline.

Why?

- High-profit jobs should not be blocked by low-profit ones.
- Doing a job late leaves room for other jobs earlier.

```
1 import java.util.*;
2 class Solution {
3     //job is a class like int
4     public class job {
5         int deadline;
6         int profit;
7         job(int deadline,int profit){
8             this.deadline = deadline ;
9             this.profit=profit ;
10        }
11    }
12    public ArrayList<Integer> jobSequencing(int[] deadline, int[] profit) {
13        // code here
14        int n= deadline.length;
15        // Step 1: Create job list
16        job[] jobs = new job[n];
17        for(int i=0;i<n;i++){
18            jobs[i] = new job(deadline[i],profit[i]);
19        }
20        // Step 2: Sort jobs by profit (descending)
21        Arrays.sort(jobs,(a,b)->b.profit-a.profit);
22        // Step 3: Find max deadline
23        int maxdeadline=0;
24        for(job job:jobs){
25            maxdeadline = Math.max(maxdeadline,job.deadline);
26        }
27        //1-based indexing
28        boolean[] slot = new boolean[maxdeadline+1]; //by default each value is false
29        int jobcount =0;
30        int maxprofit =0;
31        for(job job:jobs){
32            for(int i=job.deadline;i>0;i--){
33                if(!slot[i]){ //this means slot is empty
34                    slot[i]=true;
35                    jobcount++;
36                    maxprofit+=job.profit;
37                    break;
38                }
39            }
40        }
41        return new ArrayList<Integer>();
42    }
43 }
```



```
10     }
11 }
12 public ArrayList<Integer> jobSequencing(int[] deadline, int[] profit) {
13     // code here
14     int n= deadline.length;
15     // Step 1: Create job list
16     job[] jobs = new job[n];
17     for(int i=0;i<n;i++){
18         jobs[i] = new job(deadline[i],profit[i]);
19     }
20     // Step 2: Sort jobs by profit (descending)
21     Arrays.sort(jobs,(a,b)->b.profit-a.profit);
22     // Step 3: Find max deadline
23     int maxdeadline=0;
24     for(job job:jobs){
25         maxdeadline = Math.max(maxdeadline,job.deadline);
26     }
27     //1-based indexing
28     boolean[] slot = new boolean[maxdeadline+1]; //by default each value is false
29     int jobcount =0;
30     int maxprofit =0;
31     for(job job:jobs){
32         for(int i=job.deadline;i>0;i--){
33             if(!slot[i]){ //this means slot is empty
34                 slot[i]=true;
35                 jobcount++;
36                 maxprofit+=job.profit;
37                 break;
38             }
39         }
40     }
41     ArrayList<Integer> ans =new ArrayList<>();
42     ans.add(jobcount);
43     ans.add(maxprofit);
44     return ans;
45 }
46 }
```

 Custom Input Compile & Run Submit

[gfg](#) [chatgpt](#)

Final Interview Explanation (Perfect Answer)

"I sorted jobs by profit in descending order and scheduled each job at the latest available time slot before its deadline using a boolean slot array."

Time Complexity (Earlier / Simple Greedy)

Steps involved:

1 Create job list

→ $O(n)$

2 Sort jobs by profit

→ $O(n \log n)$

3 Find free slot for each job

java

 Copy code

```
for each job:  
    scan from deadline → 1
```

- Worst case scan = $O(n)$
- For n jobs → $O(n^2)$

◆ Total Time Complexity



5000

 Copy code

+ Ask anything

.0.



◆ Total Time Complexity

SCSS

 Copy code

$O(n \log n + n^2)$
 $\approx O(n^2)$ (dominant term)

Space Complexity (Earlier / Simple Greedy)

Extra space used:

1 Job array

java

 Copy code

```
Job[] jobs = new Job[n];
```

→ $O(n)$

2 Slot array

java



 Copy code

```
boolean[] slot = new boolean[maxDeadline + 1];
```

2 Slot array

java

Copy code

```
boolean[] slot = new boolean[maxDeadline + 1];
```

Worst case:

ini

Copy code

```
maxDeadline = n
```

→ $O(n)$

◆ Total Space Complexity

mathematica

```
 $O(n + n) = O(n)$ 
```

📌 Final Answer (Interview Ready)

📌 Final Answer (Interview Ready)

✓ Earlier Approach

- Time Complexity: $O(n^2)$
- Space Complexity: $O(n)$

🎯 One-Line Explanation

"In the earlier greedy approach, slot scanning makes time complexity quadratic, while space remains linear due to slot and job arrays."

optimised solution uses dsu approach you can study it in graph

After studying replace the blank with solution









N meetings in one room



Difficulty: **Easy**

Accuracy: **45.3%**

Submissions: **378K+**

Points: **2**

Average Time: **20m**

You are given timings of n meetings in the form of $(\text{start}[i], \text{end}[i])$ where $\text{start}[i]$ is the start time of meeting i and $\text{end}[i]$ is the finish time of meeting i . Return the **maximum** number of meetings that can be accommodated in a single meeting room, when only one meeting can be held in the meeting room at a particular time.

Note: The start time of one chosen meeting can't be equal to the end time of the other chosen meeting.

Examples :

Input: $\text{start}[] = [1, 3, 0, 5, 8, 5]$, $\text{end}[] = [2, 4, 6, 7, 9, 9]$

Output: 4

Explanation: Maximum four meetings can be held with given start and end timings. The meetings are - (1, 2), (3, 4), (5, 7) and (8, 9)

Input: $\text{start}[] = [10, 12, 20]$, $\text{end}[] = [20, 25, 30]$

Output: 1

Explanation: Only one meetings can be held with given start and end timings.

Input: $\text{start}[] = [1, 2]$, $\text{end}[] = [100, 99]$

Output: 1

gf9

Handwritten solution for the problem "N meetings in one room". The title is written in blue. The input arrays are written as:

$$\text{start}[] = [0, 3, 1, 5, 5, 8]$$
$$\text{end}[] = [5, 4, 2, 9, 7, 9]$$

To the right of the arrays is a yellow rectangle representing a meeting room. Below it, four meetings are listed with arrows pointing to their indices:

$$\begin{aligned} (1, 2) &\rightarrow 3 \\ (3, 4) &\rightarrow 2 \\ (5, 7) &\rightarrow 5 \\ (8, 9) &\rightarrow 6 \end{aligned}$$

i will try to take meeting that are ending faster

Handwritten notes on a digital whiteboard showing meeting scheduling logic:

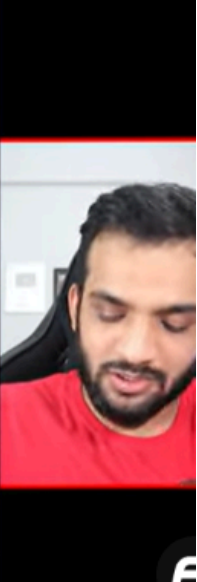
Meetings: $(1, 2, 3)$, $(3, 4, 2)$, $(0, 5, 1)$, $(5, 7, 5)$, $(5, 9, 4)$, $(8, 9, 6)$

Selection process:

- $(1, 2, 3)$ is selected (marked with a checkmark).
- $(3, 4, 2)$ is rejected (marked with an 'X') because its start time (3) is greater than the end time of the selected meeting (3).
- $(0, 5, 1)$ is rejected (marked with an 'X') because its start time (0) is less than the end time of the selected meeting (3).
- $(5, 7, 5)$ is selected (marked with a checkmark) because its start time (5) is greater than the end time of the previous selected meeting (3).
- $(5, 9, 4)$ is rejected (marked with an 'X') because its start time (5) is less than the end time of the previous selected meeting (5).
- $(8, 9, 6)$ is rejected (marked with an 'X') because its start time (8) is less than the end time of the previous selected meeting (5).

Resulting sequence: $cut = 1, 2, 3, 4$ and $free\ time = 2, 4, 7, 9$

Final sequence: $\rightarrow 1, 3, 2, 5, 6$



```
{ data {  
    start, end, pos;  
}
```

```
func (start, end, N)  
{
```

```
func (start, end, N)
```

```
{  
    Data arr [N];
```

```
    for (i = 0 → n)
```

```
    {  
        arr[i].start = start[i]
```

```
        arr[i].end = end[i]
```

```
        arr[i].pos = i+1;
```

```
    }
```

```
sort (arr, comp)
```

```
cnt = 1    freeTime = arr[0].end,    ds = {arr[0].pos}
```

```
for (i = 1 → n-1)
```

```
{  
    if (arr[i].start > freeTime)
```

```
    {  
        cnt = cnt + 1;
```

```
        freeTime = arr[i].end
```

```
        ds.add (arr[i].pos);
```

```
    }
```

```
}
```

```

b bool comp (Data val1, Data val2)
    {
        return val1.end < val2.end
    }

```

```

TC → (2N + N log N)
SC → O(3 × N) + O(N)

```

time complexity is n

(storing in

datastructure) + $n \log n$ (sorting) + n (assigning) = $2n + n \log n$

Space complexity is $O(3 \times n)$ as we are storing 3 variable for each term in datastructure and if order is asked then we will add $O(n)$ so overall space complexity is

```

SC → O(3 × N) + O(N)

```

2 Greedy Strategy (MOST IMPORTANT PART)

We use this greedy rule:

Always select the meeting that finishes earliest.

Why?

- A meeting that ends early leaves **more free time** for future meetings.
- This ensures maximum number of non-overlapping meetings.

✓ Sorting meetings by end time

java

```
Arrays.sort(data, (a, b) -> a.end - b.end);
```

This sorts meetings in **ascending order of end time**.

📌 Example:

8 What to say in an interview ?

"I used a greedy approach. By sorting meetings based on their ending time and always choosing the meeting that finishes earliest, we maximize the number of non-overlapping meetings."

✗ ACTUAL REASON FOR WRONG ANSWERS

You are initializing:

java

📄 Copy code

```
int freetime = 0;
```

This is **wrong** for this problem.

Why?

- Time values can start from **any number** (not guaranteed to start from 0).
- The **first meeting must always be selected** after sorting.

Because of this, some test cases fail.

✓ CORRECT APPROACH (Standard GFG Logic)

Step 1: Sort meetings by end time

Step 2: Select the first meeting

Step 3: Compare remaining meetings with the last selected meeting

```
1 class Solution {
2     public class meeting{
3         int start;
4         int end;
5         int position;
6         meeting(int start,int end,int position){
7             this.start = start;
8             this.end = end;
9             this.position = position;
10        }
11    }
12    public int maxMeetings(int start[], int end[]) {
13        //store start ,end ,position in datastructure
14        int n = start.length;
15        //storing meeting
16        meeting[] data = new meeting[n];
17        for(int i=0;i<n;i++){
18            data[i] = new meeting(start[i],end[i],i);
19        }
20        Arrays.sort(data,(a,b)->a.end-b.end); //if a.end-b.end > 0 then it will reverse the list means b will come first then a
21        int count=1;
22        int freetime =data[0].end;
23        for(int i=1;i<n;i++){
24            if(data[i].start>freetime){
25                count++;
26                freetime = data[i].end;
27            }
28        }
29        return count;
30    }
31 }
32 }
```

if meeting position is asked

```
ArrayList<Integer> result = new ArrayList<>();

// select first meeting
result.add(data[0].position);
int freetime = data[0].end;

for (int i = 1; i < n; i++) {
    if (data[i].start > freetime) { // STRICT >
        result.add(data[i].position);
        freetime = data[i].end;
    }
}

return result;
}
```



435. Non-overlapping Intervals

Medium

Topics

Companies

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are **non-overlapping**. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`

Output: 2

Explanation: You need to remove two `[1,2]` to make the rest of the intervals non-overlapping.

Example 3:

Input: `intervals = [[1,2],[2,3]]`

9K



153



105 Online

Java

```
1  cl
2
3  in
4  [a
5  ac
6
7
8
9  }
```

Saved

You ne

> Te

1,2 and 1,3 is overlapping at some point

1,2 and 2,3 is nonoverlapping

2,3 and 1,3 is overlapping at some point

This is same as meeting problem in that we have

We will sort it wrt end in ascending order

When we find maxnonoverlapping meeting start>=freetime so

when we subtract it from n-max nonoverlapping meeting it will give min no of overlapping meet that has to remove

```
1  class Solution {
2  |      public int eraseOverlapIntervals(int[][] intervals)
3  |      {
4  |          Arrays.sort(intervals,(a,b)->a[1]-b[1]);
5  |          //sorting meeting according to end in
6  |          ascending order
7  |          int remove =0;
8  |          int n = intervals.length;
9  |          int freetime = intervals[0][1];
10 |          int nonOver = 1;
11 |          for(int i =1;i<n;i++){
12 |              if(intervals[i][0]>=freetime){
13 |                  nonOver++;
14 |                  freetime = intervals[i][1];
15 |              }
16 |          }
17 |          remove = n-nonOver;
18 |          return remove;
19 |      }
20 |  }
```

[leetcode](#)

57. Insert Interval

Medium

Topics

Companies

Hint

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the i^{th} interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` *after the insertion*.

Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Example 1:

Input: `intervals = [[1,3],[6,9]]`, `newInterval = [2,5]`

Output: `[[1,5],[6,9]]`

Example 2:

11.6K



386



152 Online



leetcode

intervals = $\left[\begin{array}{c} \checkmark \\ [1, 2] \end{array}, \begin{array}{c} \checkmark \\ [3, 4] \end{array}, \begin{array}{c} \checkmark \\ [5, 7] \end{array}, \begin{array}{c} \checkmark \\ [8, 10] \end{array}, [12, 16] \right]$ newInterval = $[6, 8]$

$\boxed{5}$ 7
6 8
10

(1, 2) (3, 4) (5, 10)

```
func (arr [][]int, newInterval []int)
```

```
{  
    res → [][]int    i = 0
```

```
    // left
```

```
    while (i < n && arr[i][0] < newInterval[0])  
    {
```

```
        res.add(arr[i])
```

```
        i = i + 1;
```

```
    }
```

```
    while (i < n && arr[i][0] < newInterval[1])  
    {
```

```
        newInterval[0] = min(newInterval[0], arr[i][0])
```

```
        newInterval[1] = max(newInterval[1], arr[i][1]);
```

```
        i = i + 1;
```

```
    }
```

```
    res.add(newInterval)
```

```

        while (i < n)
        {
            res.add(intervals[i]);
            i = i + 1;
        }

    return res;
}

```

(1,5)(10,15)(16,20) and (6,8) are non overlapping so add it directly

(1,5)(10,15)(16,20) and (11,14) here (10,15) and (11,14) are overlapping

```

Java ▾  Auto
1 class Solution {
2     public int[][] insert(int[][] intervals, int[] newInterval) {
3         //left part
4         int i=0;
5         int n = intervals.length;
6         List<int[]> res = new ArrayList<>();
7         while(i<n && intervals[i][1]<newInterval[0]){
8             //end of left should be less than start of new interval
9             res.add(intervals[i]);
10            i++;
11        }
12        //merge part
13        while(i<n && intervals[i][0]<=newInterval[1]){
14            //ending to hai hi new interval ka
15            // agar starting se bhi km hoga ending then ye newinterval alag se add hoga then both intervals are non overlapping
16            newInterval[0] = Math.min(intervals[i][0],newInterval[0]);
17            newInterval[1] = Math.max(intervals[i][1],newInterval[1]);
18            i++;
19        }
20        res.add(newInterval);
21        //agar end start se bada hai and start bhi end se bada hai then newinterval will be non overlapping
22        //right part
23        while(i<n){
24            res.add(intervals[i]);
25            i++;
26        }
27        return res.toArray(new int[res.size()][]);
28    }
}

```

Saved

💡 Idea (3 Phases)

1. **Left part** → intervals ending **before** newInterval starts
2. **Overlapping part** → merge intervals
3. **Right part** → intervals starting **after** newInterval ends

🔄 WHILE LOOP 1 — Left Non-Overlapping Intervals

java

📄 Copy code

```
while (i < n && intervals[i][1] < newInterval[0])
```

🔍 Condition Meaning

- `intervals[i][1]` → end of current interval
- `newInterval[0]` → start of new interval

➡ If current interval ends before new interval starts,

➡ No overlap possible

📌 Why `<` ?

Because:

pgsql

📄 Copy code

```
[1,3]    newInterval = [5,7]
```

↑

↑

Ends **before** it starts → **no** overlap



WHILE LOOP 2 — Overlapping Intervals (MERGE)

java

 Copy code

```
while (i < n && intervals[i][0] <= newInterval[1])
```

Condition Meaning

- `intervals[i][0]` → start of current interval
- `newInterval[1]` → end of new interval

→ If current interval **starts before or at** newInterval end

→ Overlap exists

📌 Why `<=` ?

Because:

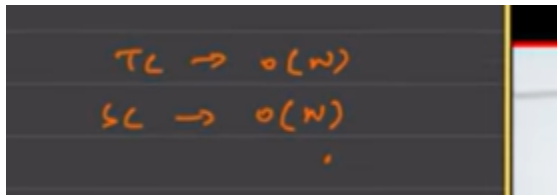
csharp

 Copy code

`[2,6]` and `[6,8]`

↑

Touching `is` also overlapping



Minimum Platforms



Difficulty: **Medium**

Accuracy: **26.84%**

Submissions: **611K+**

Points: **4**

Given arrival **arr[]** and departure **dep[]** times of trains on the same day, find the minimum number of platforms needed so that no train waits. A platform cannot serve two trains at the same time; if a train arrives before another departs, an extra platform is needed.

Note: Time intervals are in the 24-hour format (**HHMM**), where the first two characters represent hour (between 00 to 23) and the last two characters represent minutes (this will be ≤ 59 and ≥ 0). Leading zeros for hours less than 10 are optional (e.g., 0900 is the same as 900).

Examples:

Input: `arr[] = [900, 940, 950, 1100, 1500, 1800]`, `dep[] = [910, 1200, 1120, 1130, 1900, 2000]`

Output: 3

Explanation: There are three trains during the time 9:40 to 12:00. So we need a minimum of 3 platforms.

Input: `arr[] = [900, 1235, 1100]`, `dep[] = [1000, 1240, 1200]`

Output: 1

Explanation: All train times are mutually exclusive. So we need only one platform.

Menu



gf9

2 Correct Logic (VERY IMPORTANT FOR INTERVIEWS)

🔑 Key idea:

At any moment:

- If a train arrives before the previous one departs, we need one more platform
- Otherwise, a platform becomes free

🔥 Best Greedy Approach:

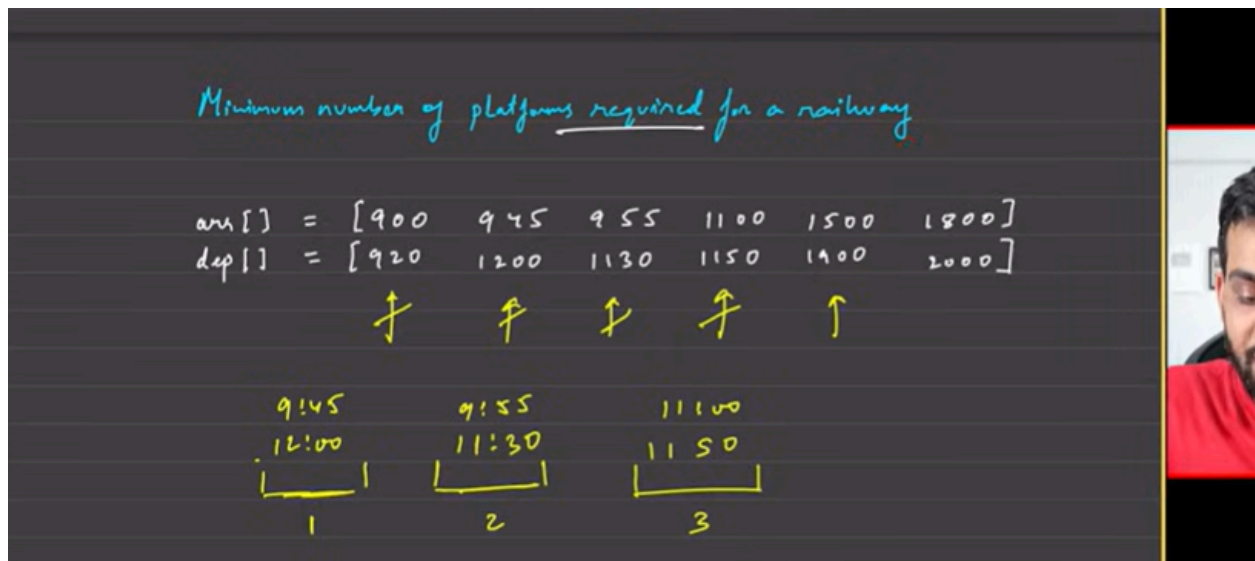
1. Sort arrival times
2. Sort departure times
3. Use two pointers

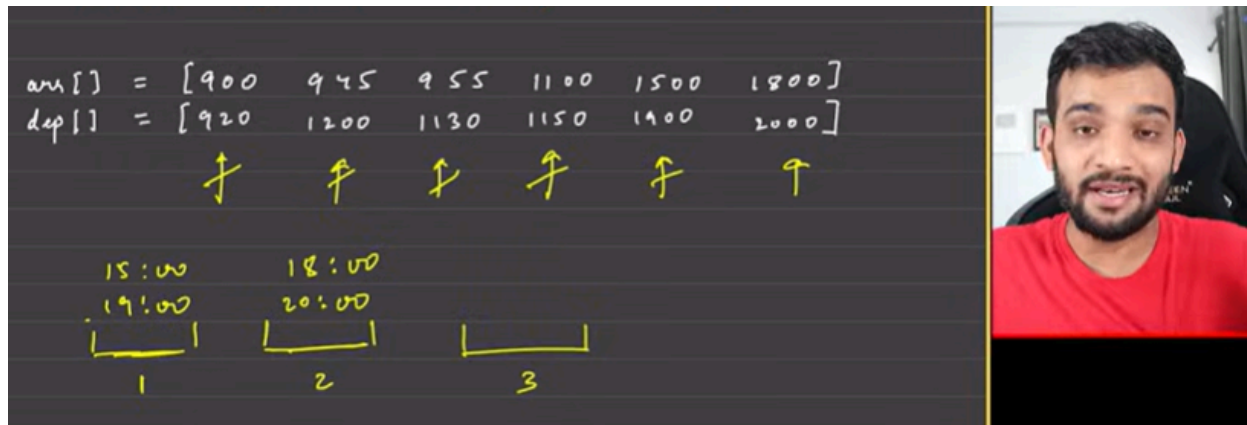
🧠 Interview explanation:

"I track the number of trains at the station at the same time.

If arrival happens before departure, platforms increase.

If departure happens first, platform frees."





we will find max intersection that will be our ans

There are 4 case arriving before departing later, arriving before departing middle, arriving middle departing later, arriving middle departing middle

So using brute force code we can say max intersection will be no of min platform required

```
import java.util.*;

class Solution {
    public int minPlatform(int[] arr, int[] dep) {
        int n = arr.length;
        int maxCount = 0;

        // Pick each train one by one
        for (int i = 0; i < n; i++) {
            int count = 1; // platform for current train

            // Check overlap with other trains
            for (int j = i + 1; j < n; j++) {
                if (arr[j] <= dep[i] && dep[j] >= arr[i]) {
                    count++;
                }
            }

            maxCount = Math.max(maxCount, count);
        }

        return maxCount;
    }
}
```

so its time complexity will be n^2 and space complexity will be 1
optimised

Minimum number of platforms required for a railway

	1	2	3	4	5	6
arr[]	900	945	955	1100	1500	1800
dep[]	920	1200	1130	1150	1900	2000

$(900, 920)$ $(945, 1200)$ $(955, 1130)$ $(1100, 1150)$ $(1500, 1900)$ $(1800, 2000)$...

$cnt = 0$

so max count is minimum no of platform

Minimum number of platforms required for a railway

	1	2	3	4	5	6
arr[]	900	945	955	1100	1500	1800
dep[]	920	1130	1150	1200	1900	2000

$cnt = 0$

```
func(arr, dep)
```

```
{  
    sort(arr)    sort(dep)
```

```
    i = 0
```

```
    j = 0
```

```
    cnt = 0
```

```
    maxCnt = 0
```

```
    while (i < n)
```

```
{
```

```
        if (arr[i] <= dep[j])
```

```
{
```

```
            cnt = cnt + 1
```

```
        }
```

YouTube

Search



Sign in

```
        cnt = 0  
        while (i < n)
```

```
        {  
            maxCnt = 0
```

```
        {
```

```
            if (arr[i] <= dep[j])
```

```
            {
```

```
                cnt = cnt + 1
```

```
                i = i + 1;
```

```
            }
```

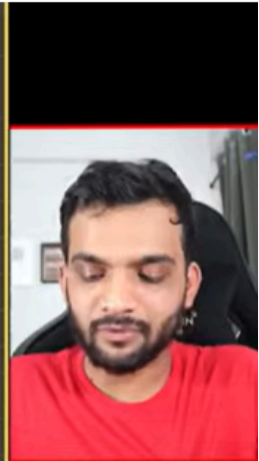
```
        }  
        else
```

```
        {
```

```
            cnt = cnt - 1
```

```
            j = j + 1;
```

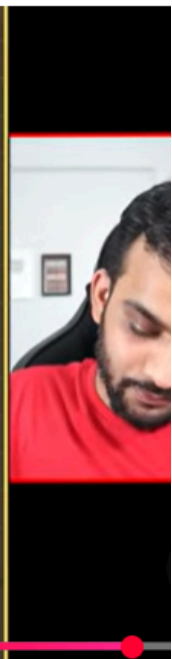
```
        }
```



```

        }
        cut = cut + 1;
        i = i + 1;
    }
    else
    {
        cut = cut - 1;
        j = j + 1;
    }
    maxCut = max(maxCut, cut);
}
return maxCut;

```



```

func(arr, dep)
{
    sort(arr)    sort(dep)    → 2N log N
    i = 0        j = 0
    cut = 0      maxCut = 0
    while (i < N) → O(N)
    {
        if (arr[i] <= dep[j])
        {
            cut = cut + 1;
            i = i + 1;
        }
    }
}

```

$T.C \rightarrow 2(N \log N + N)$
 $S.C \rightarrow O(1)$

Right now:

java

 Copy code

```
int count = 0;
```

Logically:

- First train always needs **one platform**

✓ Better:

java

 Copy code

```
int count = 1;  
int max = 1;  
int a = 1, d = 0;
```

  **Corrected & Interview-Ready Code**

```
1 class Solution {
2     public int minPlatform(int arr[], int dep[]) {
3         // code here
4         int a=1;
5         int d=0;
6         Arrays.sort(arr);
7         Arrays.sort(dep);
8         int n= arr.length;
9         int count =1;//first train always needs a platform
10        int max=1;
11        while(a<n && d<n ){
12            if(arr[a]<=dep[d]){
13                a++;
14                count++;
15            }
16            else{
17                d++;
18                count--;
19            }
20            max = Math.max(count,max);
21        }
22        return max;
23    }
24 }
25
```

20. Valid Parentheses

Solved 

Easy

 Topics

 Companies

 Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: `true`

Example 2:

Input: `s = "()[]{}"`

 27.3K



 622



 543 Onl

Resto

 1

[leetcode](https://leetcode.com/problems/valid-parentheses/)

678. Valid Parenthesis String

Medium

Topics

Companies

Hint

Given a string `s` containing only three types of characters: `'('`, `')'` and `'*'`, return `true` if `s` is **valid**.

The following rules define a **valid** string:

- Any left parenthesis `'('` must have a corresponding right parenthesis `')'`.
- Any right parenthesis `')'` must have a corresponding left parenthesis `'('`.
- Left parenthesis `'('` must go before the corresponding right parenthesis `')'`.
- `'*'` could be treated as a single right parenthesis `')'` or a single left parenthesis `'('` or an empty string `''`.

Example 1:

Input: `s = "()"`

Output: `true`

Example 2:

Input: `s = "(*)"`

6.9K



205



54 Online

Java

1

2

3

4

5

Save



[leetcode](https://leetcode.com/problems/valid-parenthesis-string/)

Valid Parenthesis String

$s = () *) * ()$

$s = (* * ($

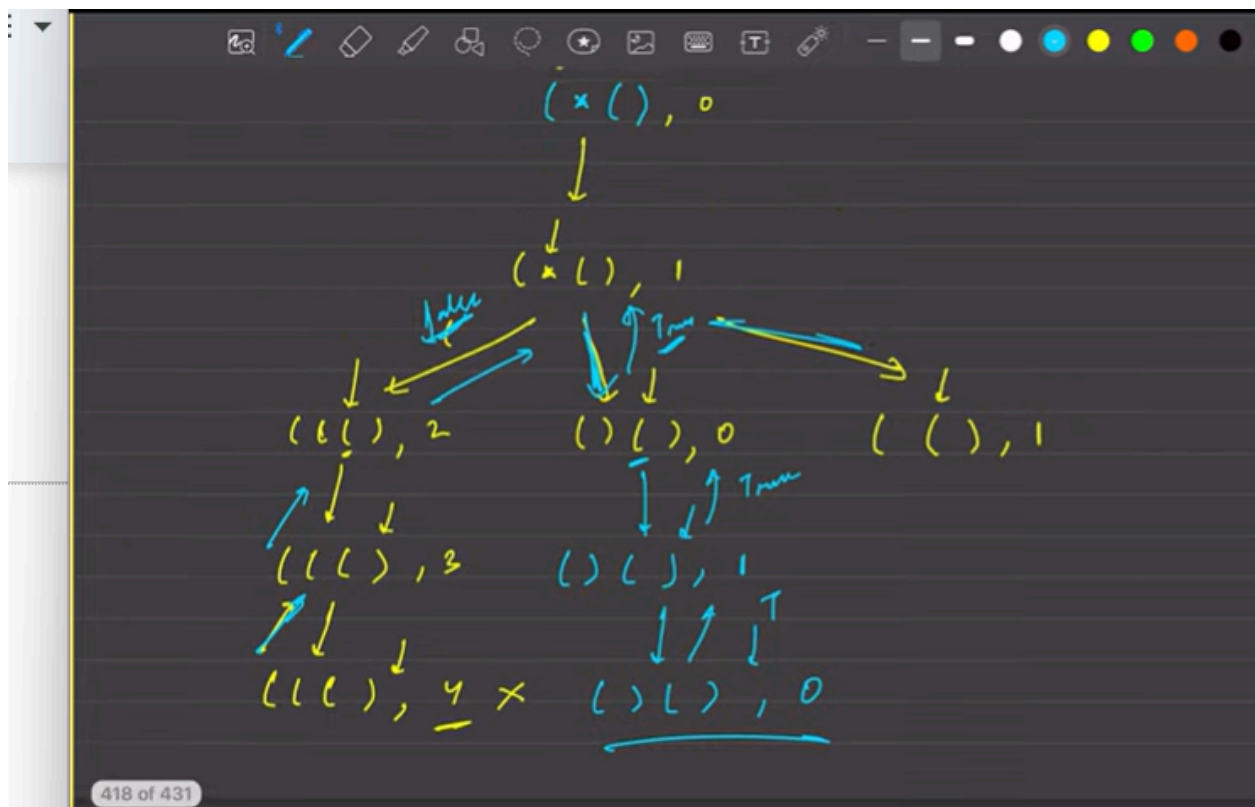
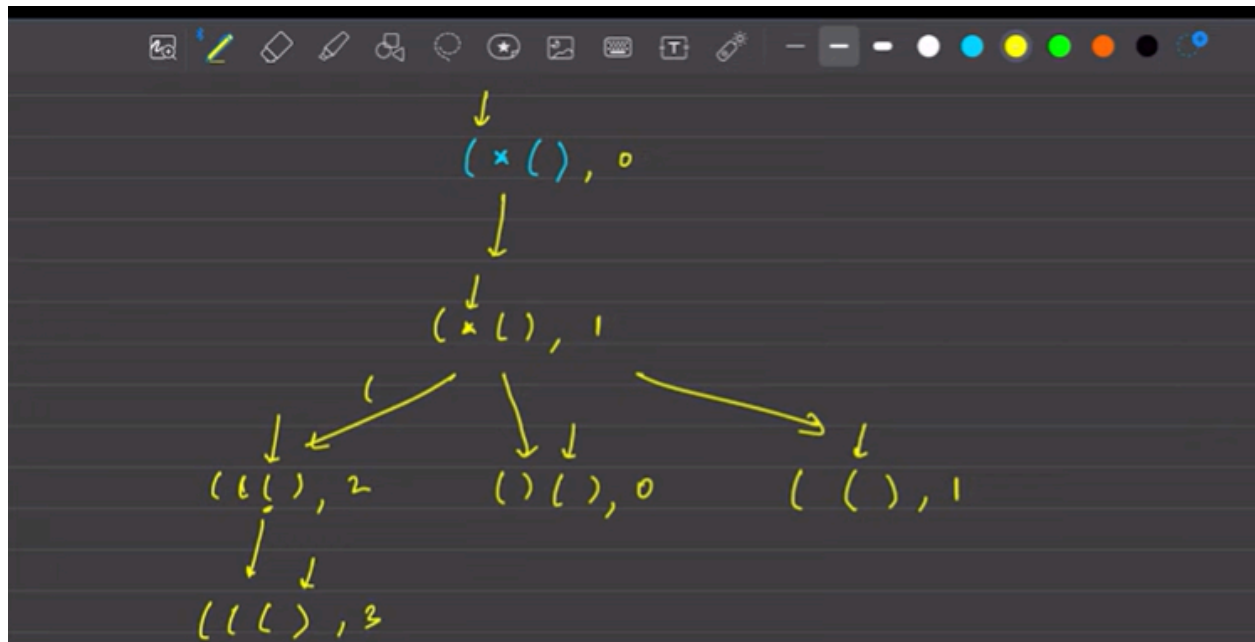
$\downarrow \downarrow \downarrow \downarrow \downarrow \downarrow$
 $s = (() ())$

$cnt = 0 \times 2 \times 2 + \underline{0}$

$\downarrow \downarrow \downarrow$
 $s = ()) ($

$cnt = 0 + \boxed{r-1}$
 < 0

if anytime count becomes 0
then that means that closing doesn't have opening
and stop there return invalid parenthesis



if (cnt < 0) return false;

if (ind == n)

return (cnt == 0);

?

if (s[ind] == '(')

return f(s, ind+1,

if (s[ind] == '(')

return f(s, ind+1, cnt+1)

if (s[ind] == ')')

return f(s, ind+1, cnt-1);

```

    }
    return (cut == 0) ;
}

if ( s[ind] == '(' )
    return f ( s, ind+1, cut+1 )

if ( s[ind] == ')' )
    return f ( s, ind+1, cut-1 ) ;

return f ( s, ind+1, cut+1 ) or f ( s, ind+1, cut-1 ) or
        f ( s, ind+1, cut )

```

for recursion time complexity we will see how many branches we are forming

Here in worst case scenario we are making 3 branches so time complexity will be 3^n and space

complexity is order of (n)

```

    TL  $\rightarrow 3^N$ 
    SC  $\rightarrow O(N)$ 

```

For optimal solution you can use dynamic programming
Below is naive approach code

```
class Solution {  
  
    public boolean isValid(String s) {  
        return f(s, 0, 0);  
    }  
  
    private boolean f(String s, int ind, int cnt) {  
        // if count becomes negative → invalid  
        if (cnt < 0) return false;  
  
        // if we reached end of string  
        if (ind == s.length()) {  
            return cnt == 0;  
        }  
  
        // if opening bracket  
        if (s.charAt(ind) == '(') {  
            return f(s, ind + 1, cnt + 1);  
        }  
  
        // if closing bracket  
        if (s.charAt(ind) == ')') {  
            return f(s, ind + 1, cnt - 1);  
        }  
    }  
}
```

 Copy code

+ Ask anything



```
// reached end of string
if (ind == s.length()) {
    return cnt == 0;
}

char ch = s.charAt(ind);

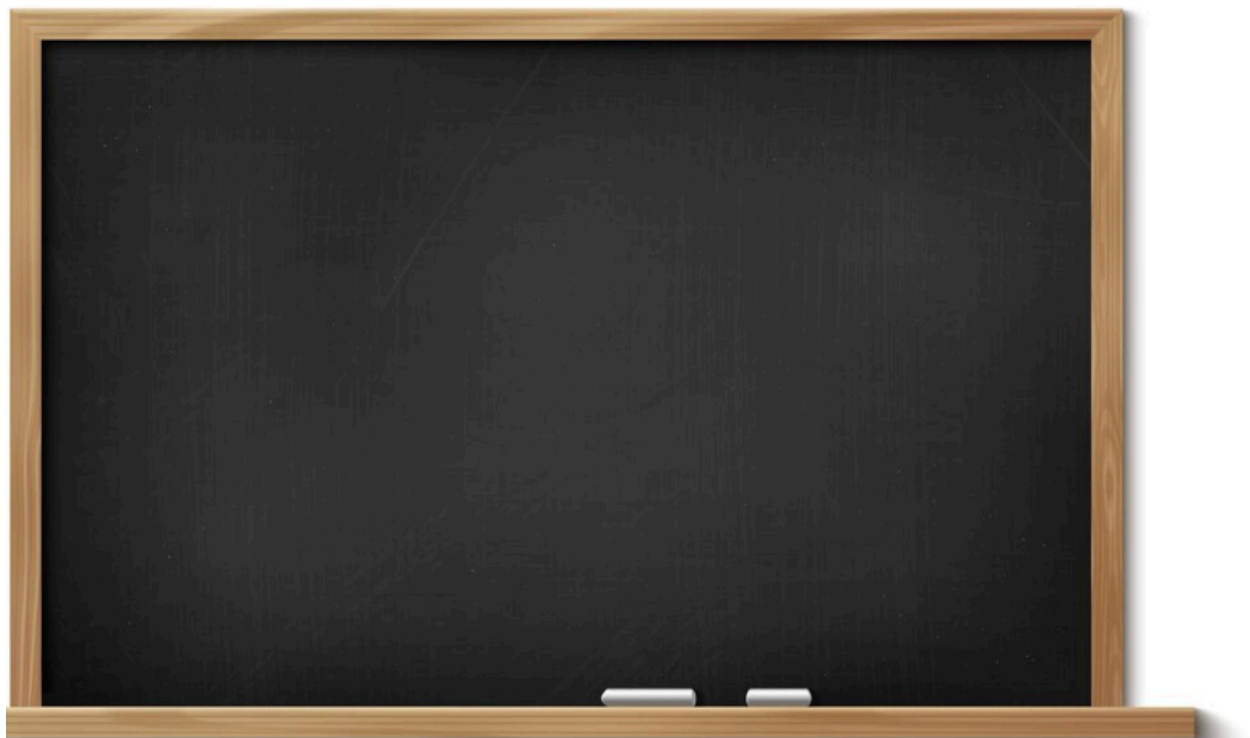
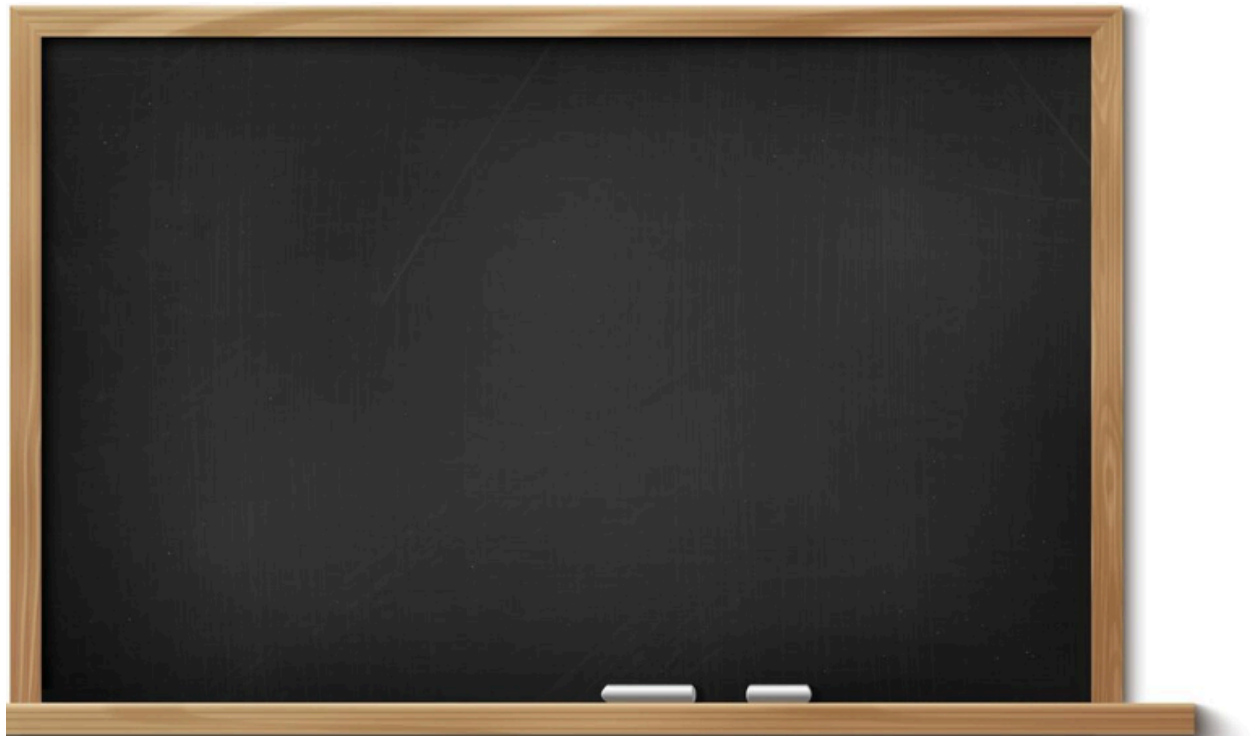
if (ch == '(') {
    return f(s, ind + 1, cnt + 1);
}

if (ch == ')') {
    return f(s, ind + 1, cnt - 1);
}

// ch == '*'
return
    f(s, ind + 1, cnt)      || // treat '*' as empty
    f(s, ind + 1, cnt + 1)  || // treat '*' as '('
    f(s, ind + 1, cnt - 1); // treat '*' as ')'
}
```

 Copy code





more optimal solution is below

```
func(s)
```

```
{  
    min = 0    max = 0
```

```
    for (i = 0 → n)
```

```
    {  
        if (s[i] == '(')
```

```
        {  
            min = min + 1
```

```
            max = max + 1
```

```
        }
```

```
    }  
    else if (s[i] == ')')
```

```
    {  
        min = min - 1
```

```
        max = max + 1;
```

```
    }
```

```
}
```

```
else
```

```
{
```

```
    min = min - 1;
```

```
    max = max + 1;
```

```
}
```

```
if (min < 0) min = 0
```

```
if (max < 0) return false;
```

va ▾  Auto

```
1  class Solution {
2      public boolean checkValidString(String s) {
3          int min=0;
4          int max =0;
5
6          for(int i=0;i<s.length();i++){
7              if(s.charAt(i)=='('){
8                  max++;
9                  min++;
10
11
12              }
13              else if(s.charAt(i)=='')){
14                  min--;
15                  max--;
16
17              }
18              else{
19                  // range lenge;
20                  min--;
21                  max++;
22              }
```

ved

```
12     }
13     else if(s.charAt(i)=='') {
14         min--;
15         max--;
16
17     }
18     else {
19         // range lenge;
20         min--;
21         max++;
22
23     }
24     if(min<0)min=0;
25     if(max<0)return false;
26 }
27 return min==0;//true if min=0 else false
28 }
29 }
```

🕒 Time Complexity

$O(n)$

- You traverse the string **once**
- Each character is processed in **constant time**
- No nested loops, no recursion

✅ Optimal

💾 Space Complexity

$O(1)$

- Only two integer variables: `min` and `max`
- No stack, no recursion, no extra data structures

✅ Constant space

.I.

jb min -1 aata hai tb hm empty set wala consider krke 0 kr dete hai

Candy

- each child $\rightarrow$ at least one candy
- children with higher rating has $>$ neighb

arr = [1 2 3]
1 + 2 + 3
= 6

arr = [1 3 2 1]
1 2 1 1
1 3 2 1
1 3 2 1 = 7

$$\begin{array}{rcl}
 \text{arr} & = & [\underline{0} \quad \underline{2} \quad \underline{4} \quad \underline{3} \quad \underline{2} \quad \underline{1} \quad \underline{1} \quad \underline{3} \quad \underline{5} \quad \underline{6} \quad \underline{4} \quad \underline{0} \quad \underline{0}] \\
 \text{left} & & 1 \quad 2 \quad 3 \quad 1 \quad 1 \quad 1 \quad 1 \quad 2 \quad 3 \quad 4 \quad 1 \quad 1 \quad 1 \\
 \text{right} & & \underline{1} \quad \underline{1} \quad \underline{4} \quad 3 \quad 2 \quad 1 \quad 1 \quad 1 \quad 1 \quad 3 \quad 2 \quad 1 \quad 1 \\
 & & 1 \quad 2 \quad 4 \quad 3 \quad 2 \quad 1 \quad 1 \quad 2 \quad 3 \quad 4 \quad 2 \quad 1 \quad 1 \quad =
 \end{array}$$

taking max of left and right will be the ans

135. Candy

Hard

Topics

Companies

There are n children standing in a line. Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

Input: `ratings = [1,0,2]`

Output: 5

Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

Java

1
2
3
4
5

Saved

[leetcode](https://leetcode.com/problems/candy/)

Left[0] will be 1 always because it doesn't have left

neighbour and $\text{right}[n-1]$ will be 1 always because it doesn't have right neighbour

```
func (ratings []int)
{
    left [n] , right [n]

    left [0] = 1    right [n-1] = 1

    for (i = 1 → n-1)
    {
        if (ratings [i] > ratings [i-1])
            left [i] = left [i-1] + 1;
        else
            left [i] = 1
    }
}
```

```
for (i = n-2; i >= 0; i--)
{
    if (ratings [i] > ratings [i+1])
        right [i] = right [i+1] + 1;
    else
        right [i] = 1;
}
```

```

sum = 0
for i = 0 → n-1
    sum = sum + max(left[i], right[i])
return sum

```

```

TC → O(3N)
SC → O(2N)

```

time complexity is $O(3N)$ as we are running two loops and space complexity is $O(2N)$ as we are using two arrays left and right

```

1  class Solution {
2      public int candy(int[] ratings) {
3          int n = ratings.length;
4          int[] left = new int[n];
5          int[] right = new int[n];
6          left[0] = 1; //as there are no neighbour in left
7          right[n-1] = 1; //as there are no neoghtbour in right
8          for(int i=1; i<n; i++){
9              if(ratings[i]>ratings[i-1]){
10                 left[i] = 1+left[i-1];
11             }
12             else{
13                 left[i] = 1;
14             }
15         }
16         for(int i=n-2; i>=0; i--){
17             if(ratings[i]>ratings[i+1]){
18                 right[i] = 1+right[i+1];
19             }
20             else{
21                 right[i] = 1;
22             }
23         }
24         int sum = 0;
25         for(int i=0; i<n; i++){
26             sum+=Math.max(left[i],right[i]);
27         }
28         return sum;
29     }

```

Saved

Ln 28, Col 20

For better soln we will remove right array and while right term we will calculate sum

$n-1$

```
else
    left[i] = 1
}
{
    cur = 1, right = 1
    for (i = n-2; i >= 0; i--)
        if (ratings[i] > ratings[i+1])
            cur = right + 1, right = cur;
        else
            cur = 1
    sum = sum + max(left[i], cur)
}
```

TC $\rightarrow O(2N)$
SC $\rightarrow O(2N)$

```

2      public int candy(int[] ratings) {
3          int n = ratings.length;
4          int[] left = new int[n];
5          left[0] = 1; // as there are no neighbour in left
6
7          for (int i = 1; i < n; i++) {
8              if (ratings[i] > ratings[i-1]) {
9                  left[i] = 1 + left[i-1];
10             }
11             else {
12                 left[i] = 1;
13             }
14         }
15         // we have removed right
16         int right = 1; // rightmost element
17         int curr = 0;
18         int sum = Math.max(right, left[n-1]);
19         for (int i = n-2; i >= 0; i--) {
20             if (ratings[i] > ratings[i+1]) {
21                 curr = 1 + right;
22                 right = curr;
23             }
24             else {
25                 curr = 1;
26                 right = curr;
27             }
28             sum = sum + Math.max(curr, left[i]);
29         }
30         return sum;
31     }

```

Saved

In 5 Col(s)

its time complexity is $2n$ and space complexity is n

Optimal solution


```

down = 1
while ( i < n { ratings[i] < ratings[i-1] }
{
    sum += down, i++, 2
}

```

```

down = 1
while ( i < n { ratings[i] < ratings[i-1] }
{
    sum += down, i++, down++;
}

```

```

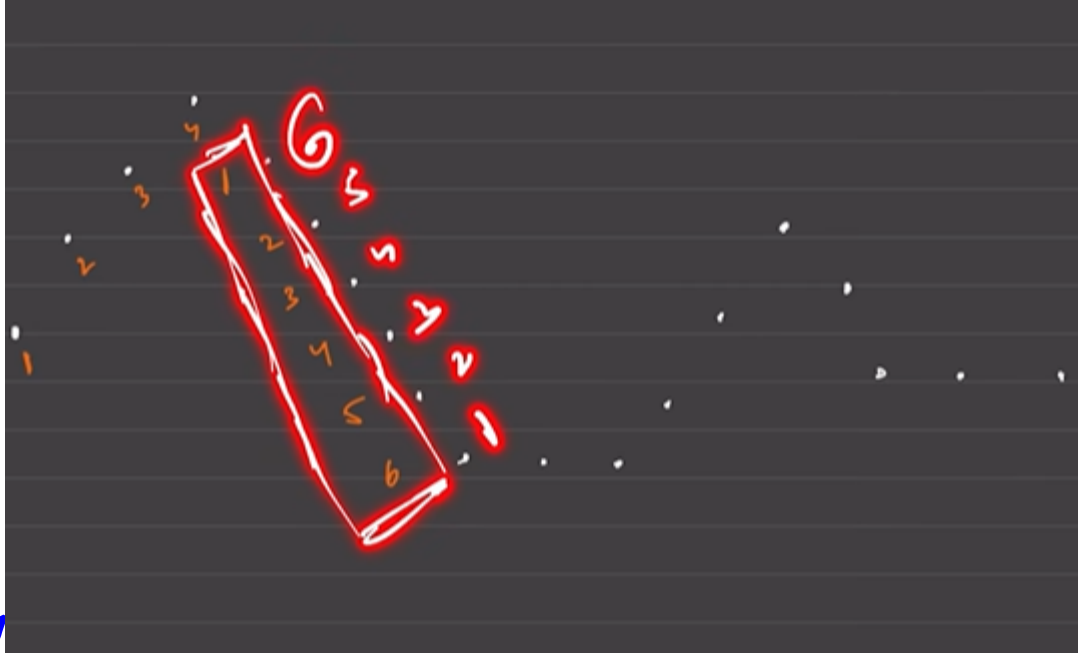
if ( down > peak )
    sum += down - peak;

return sum;

```

time

complexity is of order n and space complexity is of



oder 1

technically it should be written like the one in not in box but we have reversed it but it will still give the same sum

On reverse order when we go to peak it gives 7 it will satisfy both left and right neighbour

```

1  class Solution {
2      public int candy(int[] ratings) {
3          int i=1;
4          int sum =1;
5          int n = ratings.length;
6          while(i<n ){
7              if(ratings[i]==ratings[i-1]){
8                  sum+=1;//new ko 1 cany
9                  i++;
10                 continue;
11             }
12             int peak =1;
13             while(i<n && ratings[i]>ratings[i-1]){
14                 peak++;
15                 sum+=peak;
16                 i++;
17             }
18             int down =1;
19             while(i<n && ratings[i]<ratings[i-1]){
20                 sum+=down;
21                 down++;
22                 i++;
23             }
24             if(down>peak){
25                 sum+=down-peak;
26             }
27         }
28         return sum;

```

[leetcode](#)

Fractional Knapsack



Difficulty: **Medium**

Accuracy: **32.46%**

Submissions: **376K+**

Points: **4**

Average Time: **20m**

Given two arrays, **val[]** and **wt[]**, representing the values and weights of items, and an integer **capacity** representing the maximum weight a knapsack can hold, determine the maximum total value that can be achieved by putting items in the knapsack. You are allowed to break items into **fractions** if necessary. Return the **maximum** value as a double, rounded to 6 decimal places.



Examples :

Input: val[] = [60, 100, 120], wt[] = [10, 20, 30], capacity = 50

Output: 240.000000

Explanation: By taking items of weight 10 and 20 kg and 2/3 fraction of 30 kg. Hence total price will be $60+100+(2/3)(120) = 240$

Input: val[] = [500], wt[] = [30], capacity = 10

Output: 166.670000

Explanation: Since the item's weight exceeds capacity, we take a fraction $10/30$ of it, yielding value 166.670000.

Constraints:

$$1 \leq \text{val.size} = \text{wt.size} \leq 10^5$$

$$1 \leq \text{capacity} \leq 10^9$$

$$1 \leq \text{val}[i], \text{wt}[i] \leq 10^4$$



Fractional Knapsack

arr = [(100, 20), (60, 10), (100, 50), (200, 50)] W = 90

 5 6 2 4

50 → 100

1 → 100

10 → 100

~~50~~

~~20~~

~~100~~

(200, 50)

(100, 20)

(60, 10)

W = 20 20 40 10

we will find per kg weight and we will chose 1st who give max wight per kg

✓ If values come from variables (IMPORTANT for coding problems)

java

 Copy code

```
int a = 85;  
int b = 10;  
  
double result = (double) a / b;  
System.out.printf("%.6f", result);
```

to get ans in precision of 10^6 after decimal

✓ Option 1: Using `Math.round` (BEST)

java

 Copy code

```
double value = ((double) a / b) * 5;  
value = Math.round(value * 1_000_000.0) / 1_000_000.0;
```

Now `value` is numerically rounded to 6 decimal places and can be safely used later.

✓ Option 2: Using `BigDecimal` (MOST PRECISE)

Use this if **precision is critical** (finance, exact math).

java

 Copy code

```
import java.math.BigDecimal;
import java.math.RoundingMode;

BigDecimal value = BigDecimal.valueOf(a)
    .divide(BigDecimal.valueOf(b), 6, RoundingMode.HALF_UP)
    .multiply(BigDecimal.valueOf(5));

double result = value.doubleValue();
```

Fractional Knapsack $1 \rightarrow \frac{Val}{wt}$

arr = [(100, 20), (60, 10), (100, 50), (200, 50)] W = 90

↓ ↓ ↓ ↓

[(60, 10) , (100, 20) , (200, 50) , (100, 50)]

W = 90 80 60 10 0 totVal = 0 60 160 360 380

$1 \rightarrow \frac{100}{50} = 2 \times 10 = 20$



✖ Errors in Your Code

1. Comparator in `Arrays.sort` is wrong

java

Copy code

```
Arrays.sort(data,(a,b)->b.unit-a.unit);
```

- `b.unit - a.unit` is a `double`
- Comparator must return an `int`
- This will cause **compilation error or wrong sorting**

✔ Fix

java

Copy code

```
Arrays.sort(data, (a, b) -> Double.compare(b.unit, a.unit));
```

✓

```
Java (21) Start Timer
1 class Solution {
2     public class item{
3         int value;
4         int weight;
5         double unit;//wight per kg
6         item(int value ,int weight,double unit){
7             this.value =value;
8             this.weight = weight;
9             this.unit = unit;
10        }
11    }
12    public double fractionalKnapsack(int[] val, int[] wt, int capacity) {
13        // code here
14        int n = val.length;
15        item[] data = new item[n];
16        for(int i=0;i<n;i++){
17            double fraction =(double) val[i]/wt[i];
18            data[i] = new item(val[i],wt[i],fraction);
19        }
20        //we will arrange unit in descending order so that unit which cost more will be taken earlier
21        Arrays.sort(data,(a,b)-> Double.compare(b.unit,a.unit));
22        double sum = 0;
23        int maxcap = capacity;
24        int i=0;
25        while(maxcap>0 && i<n){
26            if(data[i].weight<=maxcap){
27                maxcap-=data[i].weight;
28                sum+=data[i].value;
29                i++;
30            }
31            else{
32                sum += data[i].unit*maxcap;
33                maxcap =0;
34            }
35        }
36        sum = Math.round(sum * 1_000_000.0) / 1_000_000.0;
37        return sum;
38    }
39 }
```

time complexity is $3n+n\log n$ and space complexity is

3n

1. Creating items array

```
java  
  
for (i = 0 to n-1)
```

Copy code

→ $O(n)$

2. Sorting by value/weight

```
java  
  
Arrays.sort(...)
```

Copy code

→ $O(n \log n)$ (dominant step)

3. Greedy selection loop

```
java  
  
while (i < n && capacity > 0)
```

Copy code

→ $O(n)$ (each item processed once)

✓ Total Time Complexity

$$O(n \log n)$$

$$O(n \log n)$$

Space Complexity

1. Item array

```
java  
  
item[] data = new item[n];
```

Copy code

→ $O(n)$ extra space

2. Variables (sum, i, capacity)

→ $O(1)$

3. Sorting (Java TimSort)

→ $O(\log n)$ auxiliary stack space

✓ Total Space Complexity

$$O(n)$$

1. Creating items array

```
java  
  
for (i = 0 to n-1)
```

→ $O(n)$

2. Sorting by value/weight

```
java  
  
Arrays.sort(...)
```

→ $O(n \log n)$ (dominant step)

3. Greedy selection loop

9f9